

Next Steps: Post-Training Production Session

RAG Cost Analysis, Multi-Identifier & Firecrawl Risk Assessment, Training Curriculum Gaps

Date: 5 July 2026 | **For:** Principal Review & Decision

Table of Contents

1. [RAG Knowledge Base — Cost Implications](#)
 2. [Multi-Identifier Contact System — Risks & Implications](#)
 3. [Firecrawl Integration — Risks & Implications](#)
 4. [Training Curriculum Gaps — Features Added Since Module Creation](#)
 5. [Consolidated Recommendations](#)
-

1. RAG Knowledge Base — Cost Implications

What RAG Does in the CRM

The RAG (Retrieval-Augmented Generation) system enhances the AI chatbot's ability to answer customer questions. Instead of relying only on keyword matching, it uses **semantic search** — understanding the *meaning* of questions, not just matching words.

How it works: 1. When a business owner adds a Q&A entry to their knowledge base, the system generates a **vector embedding** (a mathematical representation of the text's meaning) 2. When a customer sends a WhatsApp message, the system generates an embedding of that message 3. It compares the message embedding against all stored embeddings using **cosine similarity**

4. The closest match (above 0.7 threshold) is returned as the answer 5. If RAG fails, it falls back to the original keyword matching algorithm

Current Implementation Details

Parameter	Value
Embedding Model	openai/text-embedding-3-small via OpenRouter
Vector Dimensions	1,536 (VECTOR(1536) in PostgreSQL)
Chunk Size	500 characters, sentence-aware splitting
Similarity Threshold	0.7 (configurable)
Max Results	3 candidates per search
Fallback	Keyword matching (original algorithm)

Cost Breakdown

A. API Costs (Embedding Generation)

Scenario	Monthly Messages	Embedding Calls	Tokens Used	Monthly Cost
1 client, light use	500	500 search + ~50 CRUD	~275K	\$0.006
5 clients, moderate	5,000	5,000 search + ~250 CRUD	~2.6M	\$0.05
20 clients, active	50,000	50,000 search + ~1,000 CRUD	~25.5M	\$0.51
100 clients, heavy	500,000	500,000 search + ~5,000 CRUD	~252.5M	\$5.05

Verdict: API costs are negligible. Even at 100 active clients with heavy usage, embedding costs are ~\$5/month.

B. Storage Costs (Vector Data in Supabase)

This is where the real cost concern lies.

Component	Size Per Row
Vector data (1,536 × 4 bytes float32)	6,144 bytes (6 KB)
PostgreSQL tuple overhead	~28 bytes
HNSW index overhead	~6-7 KB
Other columns (chunk_text, metadata)	~500 bytes
Total per embedding row	~12.7 KB

Capacity on Supabase Free Tier (500 MB total database):

Scenario	Knowledge Entries	Chunks (avg 2/entry)	Vector Storage	% of Free Tier
1 client, 50 entries	50	100	1.2 MB	0.2%
5 clients, 50 each	250	500	6.2 MB	1.2%
20 clients, 100 each	2,000	4,000	49.6 MB	9.9%
50 clients, 100 each	5,000	10,000	124 MB	24.8%
100 clients, 200 each	20,000	40,000	496 MB	99.2%

Critical finding: At ~100 clients with 200 knowledge entries each, vector storage alone would consume the entire free tier. However, the CRM’s other tables (contacts, messages, conversations, campaigns, etc.) also need space.

Realistic free tier budget for vectors: ~100-150 MB (leaving 350-400 MB for operational data), supporting approximately **8,000-12,000 embedding rows = 40-60 clients with 100 entries each.**

C. Cheaper Alternatives Available

Model	Cost/M Tokens	Savings vs Current	Context Window	Quality
NVIDIA Llama Nemotron	FREE (\$0)	100%	131K	Good
Perplexity Embed V1	\$0.004	80%	32K	Good
Qwen3 Embedding 8B	\$0.01	50%	32K	Good

Model	Cost/M Tokens	Savings vs Current	Context Window	Quality
BAAI bge-m3	\$0.01	50%	8K	Good (multilingual)
Current: text-embedding-3-small	\$0.02	baseline	8K	Excellent

Note: Switching to a different model requires regenerating ALL existing embeddings (one-time cost) and may produce different vector dimensions, requiring a migration.

RAG Cost Recommendations

#	Recommendation	Priority	Impact
1	Keep current model — API costs are negligible (\$0.006-\$5/mo)	Do Now	Low cost
2	Monitor vector storage — Add dashboard metric for knowledge_embeddings row count	Do Now	Prevents surprise
3	Set per-client knowledge entry limit — Cap at 100 entries on Starter, 200 on Professional, 500 on Business tier	Do Now	Controls storage
4	Plan Supabase Pro upgrade at 20+ active clients (\$25/mo for 8 GB)	Plan	16x more storage
5	Evaluate NVIDIA free model when quality benchmarks are available	Later	Could eliminate API costs entirely

2. Multi-Identifier Contact System — Risks & Implications

What It Does

The multi-identifier system prepares the CRM for Meta’s upcoming changes to WhatsApp identity:

Identifier	Status	Description
Phone number (wa_id)	Active today	Current primary identifier, always present
BSUID (Business Stable User ID)	Not yet available	Meta's planned stable ID that persists across phone number changes
Username	Not yet available	User-chosen handle (like @username)

Resolution cascade: BSUID → Phone → Username → Create New Contact

Risk Assessment

Risk 1: Split-Identity Conflict (MEDIUM-HIGH)

Scenario: When Meta rolls out BSUID, a customer who changed their phone number could have: - Contact A: matched by old phone number - Contact B: matched by BSUID (which Meta associates with the new phone)

Current code behavior: The resolver checks BSUID first, finds Contact B, and reconciles by adding the phone. But Contact A (with the old phone and all its conversation history) remains orphaned.

What's missing: - No **merge logic** to combine Contact A and Contact B - No **conflict detection** to flag when BSUID and phone point to different contacts - No **admin notification** when reconciliation creates potential duplicates

Impact: Conversation history could be split across two contact records. The business owner sees an incomplete picture of the customer.

Risk 2: Race Condition on Reconciliation (LOW-MEDIUM)

Scenario: Two messages arrive simultaneously from the same customer (e.g., a message and a status update). Both trigger `resolveContactMultiId` concurrently.

Current code behavior: Both read the contact, both see missing BSUID, both try to UPDATE. The second update overwrites the first (last-write-wins), which is actually fine for adding identifiers. But if both try to INSERT a new contact, the unique index on (`account_id`, `bsuid`) would cause one to fail.

What’s missing: - No retry logic on unique constraint violations - No transaction wrapping around the read-check-update cycle - No optimistic locking

Impact: Rare edge case. Could cause a failed contact creation, but the next message would succeed.

Risk 3: Untestable Until Meta Rollout (LOW)

Scenario: BSUID and username fields exist in the schema but Meta hasn’t started sending them. We can’t test the full resolution cascade with real data.

Current mitigation: The code gracefully handles missing BSUID/username (they’re optional). The AI Playground could be used for simulated testing.

Impact: Low immediate risk. The code is forward-compatible and doesn’t break existing phone-based resolution.

Risk 4: Unique Index Edge Cases (LOW)

Scenario: Two different contacts legitimately share the same username (unlikely but possible during Meta’s rollout if usernames aren’t globally unique per business).

Current code behavior: The unique index `idx_contacts_username` ON `contacts(account_id, whatsapp_username)` WHERE `whatsapp_username IS NOT NULL` would reject the second insert.

Impact: Very low probability. Meta’s username system will likely enforce uniqueness.

Multi-Identifier Recommendations .

#	Recommendation	Priority	Effort
1	Add conflict detection — When BSUID matches Contact A but phone matches Contact B, log a system alert instead of silently reconciling	High	2-3 hours
2	Build contact merge utility — Admin tool to merge two contacts (combine conversations, deduplicate)	High	1-2 days
3	Add retry logic on unique constraint violations in <code>resolveContactMultiId</code>	Medium	1 hour
4			2 hours

#	Recommendation	Priority	Effort
	Wrap resolution in a transaction — Prevent race conditions on concurrent messages	Medium	
5	Monitor Meta announcements — BSUID/username rollout timeline affects when testing becomes possible	Ongoing	—
6	Do NOT remove the feature — The schema changes are harmless and the forward-compatibility is valuable	Keep	—

3. Firecrawl Integration — Risks & Implications

What It Does

Firecrawl is an external web scraping API that converts websites into clean, LLM-ready markdown. In the CRM, it's intended for: -
 Scraping client websites to auto-populate knowledge bases -
 Extracting product/service information for catalog sync -
 Competitive intelligence gathering

Current Implementation Status

Component	Status	Notes
FirecrawlClient class	Built	107 lines, API-only (no npm dependency)
SSRF protection	Solid	Blocks localhost, private IPs, non-HTTP protocols
Security audit document	Complete	Risk level: LOW
API route	Built	/api/scraping/firecrawl
Audit logging table	Created	firecrawl_audit_log in migration 057
Actual audit logging in code	Missing	Client code doesn't write to the audit table

Component	Status	Notes
Rate limiting	Missing	No per-account or global rate limits
FIRECRAWL_API_KEY	Not set	Feature is dormant until key is configured
UI integration	Not built	No user-facing interface to trigger scraping

Risk Assessment

Risk 1: Audit Logging Gap (MEDIUM)

Issue: The `firecrawl_audit_log` table exists in the database, and the security audit document lists “Audit logging to database” as a security measure, but the `FirecrawlClient` class **does not actually write to this table**.

Impact: If Firecrawl is activated, there would be no record of what URLs were scraped, by whom, or how much it cost. This contradicts the security audit’s claims.

Fix: Add audit logging calls in the API route handler (not the client class, since the route has access to the authenticated user context).

Risk 2: No Rate Limiting (MEDIUM)

Issue: The API route `/api/scraping/firecrawl` has no rate limiting. A user (or attacker with a valid session) could trigger unlimited scraping requests, running up Firecrawl API costs.

Impact: Potential cost overrun on the Firecrawl API. Firecrawl pricing is usage-based.

Fix: Add per-account rate limiting (e.g., 10 scrapes/hour, 50/day).

Risk 3: External API Dependency (LOW)

Issue: Firecrawl is a third-party service. If it goes down, changes pricing, or changes its API, the feature breaks.

Mitigation already in place: - API-only integration (no npm dependency = no supply chain risk) - Feature is dormant until explicitly activated - The CRM already has the web-scraper skill as an alternative

Impact: Low. The feature is optional and has alternatives.

Risk 4: AGPL-3.0 License Concern (VERY LOW)

Issue: Firecrawl’s source code is AGPL-3.0 licensed. If we were running their code, we’d need to open-source our modifications.

Mitigation: We use only the hosted API. No Firecrawl code runs in our process. The security audit correctly identifies this.

Impact: None, as long as we never install the npm package.

Firecrawl Recommendations

#	Recommendation	Priority	Effort
1	Implement audit logging — Add writes to <code>firecrawl_audit_log</code> in the API route	High	1 hour
2	Add rate limiting — 10 scrapes/hour, 50/day per account	High	1 hour
3	Keep dormant until a specific client use case requires it	Current	—
4	Consider removing if no use case emerges within 3 months — reduces attack surface	Medium	—
5	If activating , set up cost alerts on the Firecrawl dashboard	When needed	—

4. Training Curriculum Gaps — Features Added Since Module Creation

Timeline

- **Training modules generated:** 4 July 2026 (21 modules, 56,627 words)
- **Features added since:** 1 commit on 5 July 2026 (migration 057, 34 files, 3,404 lines)

Features NOT Covered in Any Training Module

The following features were built AFTER the training curriculum was generated and are completely absent from all 21 modules:

A. Ban Avoidance Engine (Critical — Affects Daily Operations)

Feature	What It Does	Who Needs to Know
24-hour window enforcement	Blocks free-form messages outside the 24h service window; requires templates	All users
Marketing frequency cap	Limits marketing templates to 2 per contact per 7 days	Campaign managers
Number warm-up tiers	Limits daily conversations based on account tier (250→1K→10K→100K)	Account admins
Quality rating auto-throttle	Reduces sending by 50% on YELLOW, pauses marketing on RED	Account admins
Template block rate monitoring	Auto-disables templates with >1.5% block rate	Campaign managers
Ban avoidance dashboard	Admin view showing all ban protection metrics	Super admins

Training impact: Users who don't understand these rules will be confused when messages are blocked. This is the **highest priority** training gap because it directly affects the user's ability to send messages.

Modules that need updating: - Module 5 (Sending Your First WhatsApp Message) — must explain 24h window - Module 9 (Broadcasting Messages) — must explain frequency caps and warm-up - Module 10 (Running Your First Campaign) — must explain template block rates - Module 12 (Your Daily CRM Routine) — must include checking quality rating - Module 16 (Advanced Campaigns) — must explain ban avoidance strategy - **New module needed:** "Keeping Your WhatsApp Number Safe" (ban avoidance deep dive)

B. RAG Knowledge Base Enhancement (Medium — Enhances Existing Feature)

Feature	What It Does	Who Needs to Know
Semantic search	AI understands question meaning, not just keywords	Knowledge base managers

Feature	What It Does	Who Needs to Know
Vector embeddings	Automatic behind-the-scenes; no user action needed	Technical admins only
Fallback to keywords	If RAG fails, original keyword matching still works	Support staff

Training impact: Module 14 (AI Chatbot) covers the knowledge base but doesn't mention semantic search or explain why the chatbot now gives better answers.

Modules that need updating: - Module 14 (AI Chatbot) — add section on “Smart Matching” explaining RAG in simple terms

C. AI Playground (Low — Power User Feature)

Feature	What It Does	Who Needs to Know
Test chatbot responses	Send test messages to see how the AI would respond	Knowledge base managers
Debug knowledge base	Verify that Q&A entries are being matched correctly	Technical admins

Training impact: No existing module covers this. It's a power-user/debugging tool.

Modules that need updating: - Module 14 (AI Chatbot) — add “Testing Your Chatbot” section - Module 21 (Scaling Your Business) — mention as an advanced tool

D. Public API & API Key Management (Low — Developer Feature)

Feature	What It Does	Who Needs to Know
Public API	External systems can read contacts, conversations, send messages	Developers/integrators
API key management	Generate, revoke, manage API keys from Settings	Account admins Developers only

Feature	What It Does	Who Needs to Know
Bearer token auth	Standard API authentication	

Training impact: This is a developer-facing feature. Most business owners won't use it directly.

Modules that need updating: - Module 15 (Connecting Your Online Store) — mention API as an integration method - Module 21 (Scaling Your Business) — add section on API integrations - **Optional new module:** "Connecting Other Systems with the API" (Level 4)

E. Multi-Identifier Contacts (Very Low — Future Feature)

Feature	What It Does	Who Needs to Know
BSUID support	Future-proofs contact matching when Meta rolls out stable IDs	No one yet
Username support	Supports WhatsApp usernames when available	No one yet
Auto-reconciliation	Automatically links new identifiers to existing contacts	Technical admins

Training impact: None needed now. These features are invisible to users until Meta activates them.

F. Firecrawl Web Scraping (Very Low — Dormant Feature)

Feature	What It Does	Who Needs to Know
Website scraping	Scrape websites to auto-populate knowledge base	Technical admins
Structured extraction	Extract specific data from web pages	Developers

Training impact: Feature is dormant (no API key configured). No training needed until activated.

Training Update Priority Matrix

Priority	Feature	Modules Affected	New Modules Needed	Effort
Critical	Ban Avoidance Engine	5, 9, 10, 12, 16	1 new module	3-4 hours
Medium	RAG Knowledge Base	14	None	1 hour
Low	AI Playground	14, 21	None	30 min
Low	Public API	15, 21	1 optional module	1-2 hours
None	Multi-Identifier	None	None	—
None	Firecrawl	None	None	—

Total estimated effort to update curriculum: 5-7 hours

5. Consolidated Recommendations

Immediate Actions (Do This Week)

#	Action	Area	Effort	Impact
1	Update training Module 5, 9, 10, 12, 16 with ban avoidance rules	Training	2-3 hrs	Users won't be confused by blocked messages
2	Create new training module: "Keeping Your WhatsApp Number Safe"	Training	1-2 hrs	Prevents account bans
3	Update training Module 14 with RAG explanation and AI Playground	Training	1 hr	Users understand improved chatbot
4	Add vector storage	RAG	1 hr	Prevents storage surprise

#	Action	Area	Effort	Impact
5	monitoring to admin dashboard Set knowledge entry limits per pricing tier (100/200/500)	RAG	30 min	Controls storage growth

Short-Term Actions (Within 2 Weeks)

#	Action	Area	Effort	Impact
6	Add conflict detection to multi-identifier resolver	Multi-ID	2-3 hrs	Prevents silent data issues
7	Build contact merge utility for admin	Multi-ID	1-2 days	Resolves duplicate contacts
8	Implement Firecrawl audit logging in API route	Firecrawl	1 hr	Closes security gap
9	Add Firecrawl rate limiting (10/hr, 50/day per account)	Firecrawl	1 hr	Prevents cost overrun
10	Update training Modules 15, 21 with API integration info	Training	1 hr	Completeness

Deferred Actions (When Triggered)

#	Action	Trigger	Area
11	Upgrade to Supabase Pro (\$25/mo)	20+ active clients	RAG
12	Evaluate NVIDIA free embedding model	Quality benchmarks published	RAG
13	Activate Firecrawl	Specific client use case	Firecrawl
14		First developer client	Training

#	Action	Trigger	Area
15	Create API integration training module	Meta BSUID rollout	Multi-ID
	Update multi-identifier training		

Cost Summary

Component	Current Monthly Cost	At 20 Clients	At 100 Clients
RAG API (embeddings)	\$0.006	\$0.51	\$5.05
RAG Storage (vectors)	\$0 (free tier)	\$0 (free tier)	\$25+ (Pro needed)
Firecrawl API	\$0 (dormant)	\$0 (dormant)	TBD if activated
Multi-Identifier	\$0 (no extra cost)	\$0	\$0
Total incremental	\$0.006/mo	\$0.51/mo	\$30.05/mo

Bottom line: The RAG system adds negligible API costs. Storage is the constraint, manageable with per-client limits and a \$25/mo Supabase upgrade at scale. Multi-identifier and Firecrawl add zero ongoing cost in their current state.

Document prepared for Principal review. All recommendations are presented for consideration — no action will be taken without endorsement.